

PROMPT MAESTRO

Plataforma de enseñanza práctica de Vibe Coding con FastAPI, IA e IA Agéntica

Backend obligatorio: Python + FastAPI + SQLAlchemy + Alembic + PostgreSQL

Frontend: React + TypeScript + Vite + Tailwind CSS

Referencia visual: <https://themewagon.github.io/elearning/>

1. Objetivo general

Desarrollar una plataforma web educativa para la enseñanza práctica de desarrollo de software y manejo de imágenes mediante Inteligencia Artificial, orientada a un curso de Vibe Coding. La plataforma debe permitir prácticas interactivas, reutilizables y escalables, incorporando IA y, cuando sea técnicamente viable, IA agéntica.

2. Objetivos específicos

La plataforma deberá permitir crear, administrar y ejecutar prácticas; incorporar prácticas de desarrollo de software, IA, procesamiento de imágenes y Vibe Coding; utilizar IA como apoyo pedagógico; incorporar agentes de IA cuando aporten valor; mantener las prácticas desacopladas y reutilizables; permitir integración con Moodle mediante iframe, API, Web Components y evaluar LTI; disponer de área pública y área privada; administrar usuarios, roles, permisos, configuraciones y catálogos; utilizar PostgreSQL; funcionar como monorepo; soportar Docker y despliegue nativo; y mantener documentación completa en docs.

3. Stack tecnológico obligatorio

Backend: Python 3.x, FastAPI, Pydantic/Pydantic Settings, SQLAlchemy 2.x y Alembic para PostgreSQL. Autenticación con JWT y hash seguro de contraseñas. Frontend: React + TypeScript + Vite + Tailwind CSS + React Router. Base de datos: PostgreSQL. IA: arquitectura desacoplada mediante puertos y adaptadores. Monorepo: pnpm/npm workspaces, Turborepo u otra alternativa justificada. Infraestructura: Docker opcional y despliegue Linux nativo.

4. Arquitectura general

Aplicar Clean Architecture y Hexagonal Architecture (Ports and Adapters) tanto en backend como frontend. Priorizar separación de responsabilidades, bajo acoplamiento, alta cohesión, testabilidad, inversión de dependencias, independencia de infraestructura, mantenibilidad y extensibilidad. La lógica de negocio no debe depender de FastAPI, SQLAlchemy, PostgreSQL, React ni proveedores concretos de IA.

5. Estructura del backend FastAPI

Utilizar una estructura similar a la siguiente, adaptándola cuando sea necesario sin romper la separación arquitectónica:

backend/
app/
domain/
entities/
value_objects/
repositories/
services/

application/
use_cases/
ports/
dto/
services/
infrastructure/
database/
repositories/
ai/
storage/
security/
external_services/
interfaces/
http/
routers/
controllers/
schemas/
dependencies/

main.py

Los routers de FastAPI deben actuar como adaptadores de entrada. Los casos de uso deben contener la lógica de aplicación. Los repositorios y servicios externos deben implementarse como adaptadores de salida.

6. FastAPI y API

Implementar una API REST documentada automáticamente mediante OpenAPI/Swagger. Utilizar APIRouter, dependency injection de FastAPI, Pydantic para validación y serialización, manejo centralizado de excepciones, respuestas HTTP coherentes, paginación, filtros y ordenamiento cuando corresponda. Los endpoints no deben contener lógica de negocio compleja.

7. Puertos y adaptadores

Definir interfaces/protocolos para repositorios, proveedor de IA, almacenamiento, evaluación y otros servicios externos. Por ejemplo: PracticeRepository, UserRepository, AIProvider, StoragePort y EvaluationPort. Los casos de uso dependerán de estas abstracciones. Los adaptadores concretos podrán utilizar SQLAlchemy/PostgreSQL, proveedores de LLM, almacenamiento local, MinIO o S3 y servicios externos.

8. Casos de uso

Implementar casos de uso explícitos como CreatePractice, UpdatePractice, PublishPractice, GetPractice, ListPractices, StartPractice, SubmitPractice, EvaluatePractice, GeneratePracticeFeedback, GenerateAIHint, ProcessImage, AnalyzeImage, CreateUser, AssignRole, AssignPermission, ManageCatalog y ManageConfiguration. Los casos de uso deben representar reglas y acciones de negocio, no simples funciones CRUD.

9. Base de datos y persistencia

Utilizar PostgreSQL. SQLAlchemy debe permanecer dentro de infrastructure. Utilizar Alembic para migraciones. Considerar entidades como users, roles, permissions, user_roles, role_permissions, practices, practice_categories, practice_tags, practice_contents, practice_evaluations, student_practice_attempts, student_progress, practice_submissions, ai_sessions, ai_messages, ai_tool_calls, configurations y audit_logs. El diseño debe permitir crecimiento futuro.

10. IA desacoplada

Crear una abstracción AIProvider con operaciones equivalentes a `generate_text`, `analyze_image` y `generate_feedback`. Implementar adaptadores para proveedores de IA sin acoplar el dominio a un proveedor. Crear MockAIAdapter para desarrollo y pruebas. La configuración del proveedor debe realizarse mediante variables de entorno.

11. IA agéntica

Implementar una primera versión de un AI Tutor Agent cuando aporte valor pedagógico. El agente debe trabajar con herramientas controladas como `get_practice`, `get_student_progress`, `analyze_submission`, `generate_hint`, `validate_answer`, `analyze_image` y `generate_feedback`. Debe tener objetivo, contexto, herramientas permitidas, límites de iteraciones/tokens, control de permisos, trazabilidad, manejo de errores y protección contra ciclos infinitos. No debe ejecutar acciones administrativas o destructivas sin autorización explícita.

12. Frontend

Utilizar React, TypeScript, Vite, Tailwind CSS y React Router. Aplicar Clean Architecture también en frontend. Separar domain, application, infrastructure, presentation y shared. Las llamadas HTTP deben estar aisladas en adaptadores o servicios de infraestructura. Los componentes visuales no deben contener lógica de negocio compleja.

13. Diseño visual

Tomar como referencia visual el sitio <https://themewagon.github.io/elearning/>. Crear un diseño propio inspirado en un portal e-learning moderno, sin copiar contenido propietario. Incluir header, navegación, hero, catálogo de prácticas, categorías, tarjetas, detalle de práctica, progreso, perfil y footer.

14. Área pública

Implementar inicio, catálogo de prácticas, categorías, detalle de práctica, información de la plataforma y login. Permitir filtrar por categoría, nivel, tecnología, tipo de práctica, IA e imágenes.

15. Área privada de administración

Crear un panel protegido con dashboard, prácticas, categorías, usuarios, roles, permisos, configuraciones, IA y auditoría. Implementar RBAC. Roles iniciales sugeridos: ADMIN, TEACHER, STUDENT y CONTENT_MANAGER. Permisos de ejemplo: `practice:create`, `practice:read`, `practice:update`, `practice:delete`, `practice:publish`, `user:create`, `user:read`, `user:update`, `user:delete`, `configuration:read` y `configuration:update`.

16. Modelo reutilizable de prácticas

Diseñar las prácticas como entidades configurables con campos equivalentes a `id`, `title`, `slug`, `description`, `objectives`, `instructions`, `category`, `difficulty`, `estimatedTime`, `technologies`, `type`, `content`, `evaluation`, `aiConfiguration`, `embeddingConfiguration`, `status` y `metadata`. Agregar nuevas prácticas no debe requerir modificar el núcleo de la plataforma.

17. Prácticas iniciales

Crear cuatro prácticas iniciales basadas en las guías proporcionadas dentro del proyecto: dos de desarrollo de software y dos de manejo de imágenes. Las prácticas de software deben abordar Vibe Coding, IA aplicada al desarrollo, resolución de problemas, análisis y modificación de código y evaluación. Las de imágenes deben abordar análisis, generación o transformación, interpretación y uso de IA. Los contenidos concretos deben derivarse de las guías existentes.

18. Seeds

Crear seeds idempotentes para usuarios, roles, permisos, categorías, configuraciones y las cuatro prácticas. El seed debe poder ejecutarse repetidamente sin duplicados. Crear un usuario administrador de desarrollo y documentar sus credenciales de ejemplo.

19. Flujo de ejecución de una práctica

Implementar el flujo: seleccionar práctica → leer objetivos → iniciar → recibir instrucciones → realizar actividad → enviar resultado → evaluación → feedback → sugerencias de IA → registrar progreso. Mostrar estado, progreso, resultado, retroalimentación e intentos.

20. Manejo de imágenes

Definir un StoragePort con operaciones upload, delete y get_url. Permitir adaptadores LocalStorageAdapter, MinIOAdapter y S3Adapter. Validar MIME, extensión, tamaño y seguridad de archivos. Las prácticas de imágenes deben poder ser ejecutadas independientemente del proveedor de almacenamiento.

21. Compatibilidad con Moodle

Diseñar las prácticas para poder reutilizarse fuera del portal principal. Implementar una vista de embebido independiente, por ejemplo /practices/software-01/embed. Documentar integración mediante iframe y API. Evaluar Web Components y LTI como evolución futura. El contenido embebido debe evitar depender innecesariamente del layout administrativo del portal.

22. Monorepo

Usar una estructura similar a:

vibe-coding-platform/
apps/
backend/
frontend/
packages/
shared/
types/
ui/
config/
database/
docs/
docker/
scripts/

.env.example

docker-compose.yml

deploy.yml

README.md

Seleccionar pnpm workspaces, npm workspaces, Turborepo u otra alternativa y justificar la decisión.

23. Despliegue con Docker

Permitir ejecutar frontend, backend y PostgreSQL mediante Docker. Incluir Dockerfiles, docker-compose.yml y configuración mediante .env. Debe existir un flujo simple como docker compose up -d. El backend FastAPI debe ejecutarse mediante un servidor ASGI apropiado, por ejemplo Uvicorn/Gunicorn según el entorno.

24. Despliegue sin Docker

Permitir ejecución directa sobre Linux con Python, entorno virtual, PostgreSQL, Node.js, Nginx y un gestor de procesos apropiado. Documentar instalación, configuración, migraciones Alembic, seeds, build del frontend, ejecución de FastAPI, Nginx y gestión del proceso.

25. Selección del modo de despliegue

Permitir seleccionar entre DOCKER y NATIVE mediante configuración, por ejemplo `DEPLOY_MODE=docker` o `DEPLOY_MODE=native`, y/o `deploy.yml`. Evitar duplicación innecesaria de configuraciones.

26. Variables de entorno

Crear `.env.example`. Incluir variables equivalentes a `NODE_ENV/ENVIRONMENT`, `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRE_MINUTES`, `AI_PROVIDER`, `AI_API_KEY`, `FRONTEND_URL`, `BACKEND_URL`, `STORAGE_PROVIDER`, `STORAGE_PATH` y `CORS_ORIGINS`. Nunca almacenar secretos reales en el repositorio.

27. Seguridad

Implementar JWT, hash seguro de contraseñas, RBAC, validación Pydantic, CORS configurable, rate limiting cuando corresponda, protección de endpoints administrativos, validación de archivos, límites de tamaño, sanitización, auditoría y controles para las herramientas del agente. Aplicar principio de mínimo privilegio.

28. Testing

Backend: pytest, pruebas unitarias de dominio y casos de uso, pruebas de adaptadores, repositorios y API. Frontend: pruebas de componentes y lógica crítica. IA: mocks y adaptadores simulados. Las pruebas deben poder ejecutarse sin consumir un proveedor externo de IA.

29. Documentación

Crear docs/ con `architecture.md`, `installation.md`, `configuration.md`, `deployment.md`, `docker.md`, `native-deployment.md`, `database.md`, `api.md`, `authentication.md`, `authorization.md`, `ai.md`, `ai-agents.md`, `practices.md`, `moodle-integration.md`, `development.md`, `testing.md` y `troubleshooting.md`. Documentar también decisiones arquitectónicas importantes.

30. README principal

El README debe incluir descripción, características, arquitectura, stack tecnológico, requisitos, instalación rápida, Docker, despliegue nativo, variables de entorno, migraciones, seeds, ejecución, prácticas, IA, agente, Moodle, testing y documentación.

31. Calidad de código

Backend: Python moderno, type hints, Ruff/Flake8, Black o Ruff formatter, principios SOLID, Clean Code y separación estricta de capas. Frontend: TypeScript estricto, ESLint, Prettier y componentes reutilizables. Evitar duplicación, lógica de negocio en routers, acceso directo a SQL desde casos de uso y secretos hardcodeados.

32. CI/CD

Preparar pipeline con `Install` → `Lint` → `Test` → `Build` → `Deploy`. Contemplar Docker y Native deployment mediante configuración. El pipeline debe ejecutar migraciones de forma controlada y nunca exponer secretos.

33. Experiencia de aprendizaje

La plataforma debe simular una experiencia interactiva: contenido → actividad → interacción → tutor IA/agente → evaluación → feedback → progreso. La IA debe formar parte del proceso pedagógico y no ser únicamente un chatbot independiente.

34. Extensibilidad

La arquitectura debe permitir incorporar nuevas prácticas, tipos de prácticas, proveedores de IA, agentes, modelos de evaluación, almacenamiento, catálogos, roles, mecanismos Moodle y tipos de contenido sin modificar significativamente el núcleo.

35. Entregables

Entregar monorepo con FastAPI backend, React frontend, PostgreSQL, migraciones, seeds, panel administrativo, usuarios, roles, permisos, catálogo, prácticas, progreso, evaluación, IA, primer agente, manejo de imágenes, integración/embebido para Moodle, Docker, despliegue nativo y documentación completa.

36. Forma de trabajo del agente de coding

El agente debe trabajar incrementalmente: analizar guías → arquitectura → base de datos → monorepo → backend FastAPI → autenticación/autorización → prácticas → frontend → administración → IA → agente → imágenes → Moodle → Docker → despliegue nativo → testing → documentación. Después de cada fase debe ejecutar pruebas y verificar que el proyecto siga funcionando.

37. Regla fundamental sobre las guías

Antes de implementar las prácticas, analizar todos los documentos, guías, ejemplos y recursos proporcionados dentro del proyecto. Las prácticas deben derivarse de ese material. No sustituir contenidos específicos por ejemplos genéricos cuando las guías ya los definan. Las decisiones no especificadas deben ser técnicamente razonables y quedar documentadas.

38. Resultado esperado

Construir una plataforma educativa funcional de Vibe Coding basada en IA, con backend FastAPI y arquitectura limpia/hexagonal, prácticas interactivas reutilizables, IA y agentes, administración de usuarios y contenidos, PostgreSQL, integración con Moodle y despliegue mediante Docker o directamente sobre Linux. Debe poder crecer desde cuatro prácticas iniciales hacia un catálogo amplio. Priorizar mantenibilidad, reutilización, seguridad, experiencia de usuario, testabilidad y extensibilidad.